

Resolución de Actividades - Programación II

Este documento contiene las resoluciones a las actividades prácticas y al TP N°7 sobre Clases, Constructores y Destructores.

2. TP N°7 - Revisión de Clases. Constructores y destructores

A- Respuestas a las actividades teóricas

1. **¿Qué es el encapsulamiento? ¿Cuáles son los niveles de acceso a miembros de la clase?**
 - El encapsulamiento es una característica relevante de la programación orientada a objetos que trata de que, en lo posible, solo pueda accederse a una variable de clase a través de un método de dicha clase.
 - Los niveles de acceso a los miembros de la clase son:
 - **Acceso público (public):** Permite que un miembro de clase sea accedido directamente a través del operador punto. Reglamentariamente, los métodos de una clase tienen este nivel de acceso.
 - **Acceso privado (private):** Permite la ocultación de ciertos miembros de la clase y restringe el acceso a dichos miembros a solo otros miembros de la misma clase. Solo puede accederse a un miembro privado mediante un miembro público que acceda directamente a él. Este es el nivel de acceso por defecto de los miembros de una clase en C++.
2. **¿Qué son los getters y setters? ¿Cuál es su propósito principal? ¿Cuándo es conveniente usarlos?**
 - Los métodos Get y Set son métodos que reglamentariamente se definen para el acceso a una variable privada de la clase.
 - Su propósito principal es obtener el estado actual (valor) de la variable privada (mediante el método Get) y modificar dicho valor directamente (mediante el método Set).

- Es conveniente usarlos cuando se necesita acceder o modificar una variable con nivel de acceso privado, dado que un método Get devuelve el valor actual de la variable privada y un método Set modifica directamente el valor de la variable privada.

3. **¿Cuál es la diferencia entre un constructor y un destructor? ¿Cuándo se ejecutan cada uno?**

- **Constructor:** Es un método especial cuyo objetivo fundamental es inicializar los atributos del objeto creado. **Ejecución:** Es el primer método que se ejecuta cuando se crea un objeto. Se ejecuta inicialmente y en forma automática inmediatamente luego de crear un objeto.
- **Destructor:** Es un método especial que elimina el objeto. **Ejecución:** Es el último método que se ejecuta cuando se elimina el objeto y en forma automática. Se ejecuta una única vez.

4. **Realizar un cuadro comparativo entre las características de constructores y destructores**

Característica	Constructor	Destructor
Momento de ejecución	Es el primer método que se ejecuta.	Es el último método que se ejecuta.
Forma de ejecución	Se llama automáticamente al crear un objeto.	Se ejecuta en forma automática al eliminar el objeto.
Frecuencia	No puede ser llamado nuevamente luego de crear el objeto.	Se ejecuta una única vez.
Nomenclatura	Tiene el mismo nombre de la clase.	Tiene el mismo nombre de la clase más el caracter ~ al principio.

Característica	Constructor	Destructor
Retorno de datos	No puede retornar datos.	No puede retornar datos.
Obligatoriedad	Imposible olvidarse llamarlo.	No es obligatorio.

5. **Diferencia entre definir una función miembro completamente dentro de la definición de la clase o fuera de ella.**

- La única diferencia radica en optimizaciones del compilador.
- **Definición dentro de la clase (Función inline):** La función es considerada automáticamente una función miembro inline. El compilador "expande" el cuerpo de la función en cada punto de llamada, evitando la sobrecarga asociada a las llamadas a funciones tradicionales.
- **Definición fuera de la clase:** La función es una función miembro normal (no inline). Se define fuera ayudándose del operador de resolución de ámbito ::.

B- Resolución de las actividades planteadas en el TP N°7 (Código)

1. Clase Alumno

```
#include <iostream>
#include <string>
using namespace std;

class Alumno {
private:
    string nombre;
    int edad;
public:
    Alumno();
    void imprimir();
    void verificarMayoriaEdad();
};

Alumno::Alumno() {
    cout << "--- Carga de Alumno ---" << endl;
    cout << "Ingrese el nombre: ";
    getline(cin, nombre);
```

```

        cout << "Ingrese la edad: ";
        cin >> edad;
    }

    void Alumno::imprimir() {
        cout << "\nNombre del alumno: " << nombre << endl;
        cout << "Edad: " << edad << endl;
    }

    void Alumno::verificarMayoriaEdad() {
        if (edad >= 18) {
            cout << "El alumno es mayor de edad." << endl;
        } else {
            cout << "El alumno no es mayor de edad." << endl;
        }
    }

    int main() {
        Alumno alu;
        alu.imprimir();
        alu.verificarMayoriaEdad();
        return 0;
    }

```

2. Clase Empleado

```

#include <iostream>
#include <string>
using namespace std;

class Empleado {
private:
    string nombre;
    float sueldo;
public:
    Empleado();
    void imprimir();
    void verificarImpuestos();
};

Empleado::Empleado() {
    cout << "--- Carga de Empleado ---" << endl;
    cout << "Ingrese nombre del empleado: ";
    getline(cin, nombre);
    cout << "Ingrese sueldo: ";

```

```

        cin >> sueldo;
    }

    void Empleado::imprimir() {
        cout << "\nEmpleado: " << nombre << endl;
        cout << "Sueldo: $" << sueldo << endl;
    }

    void Empleado::verificarImpuestos() {
        if (sueldo > 3000) {
            cout << "Debe pagar impuestos." << endl;
        } else {
            cout << "No debe pagar impuestos." << endl;
        }
    }

    int main() {
        Empleado emp;
        emp.imprimir();
        emp.verificarImpuestos();
        return 0;
    }

```

3. Clase Operaciones

```

#include <iostream>
using namespace std;

class Operaciones {
private:
    int valor1;
    int valor2;
public:
    Operaciones(int v1, int v2);
    void sumar();
    void restar();
    void multiplicar();
    void dividir();
};

Operaciones::Operaciones(int v1, int v2) {
    valor1 = v1;
    valor2 = v2;
}

```

```

void Operaciones::sumar() {
    cout << "Suma: " << (valor1 + valor2) << endl;
}

void Operaciones::restar() {
    cout << "Resta: " << (valor1 - valor2) << endl;
}

void Operaciones::multiplicar() {
    cout << "Multiplicacion: " << (valor1 * valor2) << endl;
}

void Operaciones::dividir() {
    if (valor2 != 0) {
        cout << "Division: " << (static_cast<float>(valor1) /
valor2) << endl;
    } else {
        cout << "Division: No es posible dividir por cero." <<
endl;
    }
}

int main() {
    int num1, num2;
    cout << "Ingrese el primer valor: ";
    cin >> num1;
    cout << "Ingrese el segundo valor: ";
    cin >> num2;

    Operaciones op(num1, num2);

    cout << "\nResultados:" << endl;
    op.sumar();
    op.restar();
    op.multiplicacion();
    op.dividir();

    return 0;
}

```

4. Clase Sumatoria

```

#include <iostream>
using namespace std;

```

```

class Sumatoria {
private:
    int suma;
public:
    Sumatoria();
    ~Sumatoria();
};

Sumatoria::Sumatoria() {
    int valor;
    suma = 0;
    cout << "--- Calculadora de Sumatoria ---" << endl;
    do {
        cout << "Ingrese un valor entero (Ingrese 0 para
finalizar): ";
        cin >> valor;
        suma += valor;
    } while (valor != 0);
}

Sumatoria::~~Sumatoria() {
    cout << "\n[Ejecutando Destructor] La suma de los valores
ingresados es: " << suma << endl;
}

int main() {
    Sumatoria sum;
    return 0;
}

```

5. Clase Estudiante

```

#include <iostream>
#include <string>
using namespace std;

class Estudiante {
private:
    string nombre;
    int edad;
    float calificacion;

public:
    void setNombre(string n);
    void setEdad(int e);

```

```

    void setCalificacion(float c);

    string getNombre();
    int getEdad();
    float getCalificacion();
};

void Estudiante::setNombre(string n) { nombre = n; }
void Estudiante::setEdad(int e) { edad = e; }
void Estudiante::setCalificacion(float c) { calificacion = c; }

string Estudiante::getNombre() { return nombre; }
int Estudiante::getEdad() { return edad; }
float Estudiante::getCalificacion() { return calificacion; }

int main() {
    Estudiante est;

    est.setNombre("Juan Perez");
    est.setEdad(17);
    est.setCalificacion(8.5);

    cout << "--- Datos del Estudiante ---" << endl;
    cout << "Nombre: " << est.getNombre() << endl;
    cout << "Edad: " << est.getEdad() << " años" << endl;
    cout << "Calificacion: " << est.getCalificacion() << endl;

    return 0;
}

```